Lectura y escritura de ficheros en PHP

Apuntes explicativos y prácticos - DWES, 2º DAW

Guía para entender y practicar el trabajo con ficheros de texto: cómo guardar registros, cómo leerlos, cómo convertirlos en datos y cómo mostrarlos. Está construida a partir de dos ejercicios tipo: **guardar las entradas vendidas** (examen de venta de entradas, ejercicio 3) y el **juego de adivinar capitales** (fichero `id;país;capital`, botones Validar y Pasar).

Apartado	Contenido
1. Idea general	Qué es un fichero de texto con registros y cómo se organiza el trabajo
2. Rutas y preparación	Dónde se guarda el fichero, `__DIR__`, codificación UTF-8
3. Escribir	Añadir al final, sobrescribir, `fopen` con modos, errores
4. Leer	`file()`, `fgets`, `file_get_contents`, trampas con `feof` y saltos de línea
5. De línea a datos	`explode`, `implode`, arrays asociativos y objetos
6. Modificar y borrar	Reescribir el fichero, borrar un registro, id autoincremental
7. Errores y excepciones	Qué comprobar y cómo avisar al usuario
8. Ejercicio A	Entradas vendidas: guardar y mostrar la tabla (examen)
9. Ejercicio B	Juego de capitales completo con Validar y Pasar
10. Prácticas propuestas	Enunciados con pistas, y dos con solución
11. Errores típicos y checklist	Por qué no se guarda / no se lee y lista de comprobación

1. La idea general

Un **fichero de texto con registros** es la "base de datos" más sencilla: cada **línea** es un **registro** y los **campos** de cada registro se separan con un carácter que no aparezca en los datos (aquí, `;`).

```
entradas.txt                                                    paises.txt
José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46      1;España;Madrid
Ana López;Mayor de 60;23/11/2024;2;Ninguno;16.00                            2;Portugal;Lisboa
                                                                           3;Italia;Roma
```

Casi cualquier ejercicio de ficheros se resuelve con el mismo ciclo de tres pasos:

Paso	Qué haces	Funciones habituales
1. Escribir	Convertir un dato (objeto, array) en una línea de texto y añadirla al fichero.	`implode(';', ...)`, `file_put_contents(..., FILE_APPEND)`, `fwrite`
2. Leer	Traer las líneas del fichero a un array de PHP.	`file()`, `fgets`, `file_get_contents`
3. Interpretar y mostrar	Trocear cada línea en campos, crear arrays u objetos y pintarlos en HTML.	`explode(';', ...)`, `foreach`, `htmlspecialchars`

Chuleta rápida: quiero... -> uso

Quiero...	Uso
Añadir una línea al final	`file_put_contents($ruta, $linea . PHP_EOL, FILE_APPEND | LOCK_EX);`
Sobrescribir el fichero entero	`file_put_contents($ruta, $texto, LOCK_EX);`
Leer todas las líneas a un array	`$l = file($ruta, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);`
Leer todo como un único texto	`$t = file_get_contents($ruta);`
Leer línea a línea (ficheros grandes)	`$f = fopen($ruta,'r'); while (($l = fgets($f)) !== false) {...} fclose($f);`
Saber si existe / se puede leer / escribir	`is_file($ruta)`, `is_readable($ruta)`, `is_writable($ruta)`
Contar registros	`count(file($ruta, FILE_SKIP_EMPTY_LINES))`
Trocear una línea en campos	`$campos = explode(';', $linea);`
Unir campos en una línea	`$linea = implode(';', $campos);`
Borrar un registro	Leer todas las líneas, quitar la que sobra y **reescribir** el fichero.
Borrar el fichero	`unlink($ruta);`

2. Rutas y preparación

2.1 Dónde se guarda el fichero

- Una ruta **relativa** (`'entradas.txt'`) se interpreta desde el directorio de trabajo, que normalmente es la carpeta del script que ha recibido la petición. Si incluyes ficheros desde otras carpetas puede cambiar y dar errores difíciles de ver.
- Para evitar sorpresas usa siempre `__DIR__`, que es la carpeta del fichero PHP actual: `__DIR__ . '/entradas.txt'`.
- Subcarpetas: `__DIR__ . '/datos/entradas.txt'`. La carpeta debe existir (`mkdir`) y Apache debe poder escribir en ella (en XAMPP con Windows normalmente no hay problema).
- Define la ruta **una sola vez** como constante y úsala en todas las funciones: `const FICHERO = __DIR__ . '/entradas.txt';`.
- Cualquier fichero dentro de `htdocs` se puede abrir desde el navegador (`http://localhost/.../entradas.txt`). Para ejercicios no importa; en un proyecto real los datos irían fuera de la carpeta pública.

2.2 Codificación y saltos de línea

- Guarda los ficheros de datos en **UTF-8 sin BOM** (en el Bloc de notas: Guardar como -> Codificación UTF-8). Si no, las tildes y la ñ salen como `Ã¡` o ■. El BOM es un carácter invisible al principio que rompe el primer campo.
- Pon `<meta charset="UTF-8">` en el HTML.
- En Windows un salto de línea son dos caracteres (`\r\n`); en Linux, uno (`\n`). Al escribir usa `PHP_EOL`, y al leer elimina siempre los saltos con `FILE_IGNORE_NEW_LINES` o `rtrim($linea, "\r\n")`.

2.3 El separador y los datos del usuario

Si un usuario escribe `;` o pulsa Intro dentro de un campo (por ejemplo en un textarea), la línea se corrompe: aparece un campo de más o una línea partida en dos. Limpia **siempre** los textos libres antes de guardarlos:

```
/** Quita del texto los caracteres que romperían el formato del fichero. */
function limpiarCampo(string $texto): string {
    return trim(str_replace([';', "\r", "\n"], ' ', $texto));
}
```

3. Escribir en ficheros

3.1 La forma corta: file_put_contents

Abre, escribe y cierra en una sola instrucción. Devuelve el número de bytes escritos o `false` si falla. Es la opción más cómoda para los ejercicios.

```
const FICHERO = __DIR__ . '/entradas.txt';
$linea = 'José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46';

// AÑADIR al final (si el fichero no existe, lo crea)
$bytes = file_put_contents(FICHERO, $linea . PHP_EOL, FILE_APPEND | LOCK_EX);
if ($bytes === false) {                                  // === false: escribir 0 bytes no es un error
    throw new RuntimeException('No se pudo escribir en el fichero.');
}

// SOBRESCRIBIR (sin FILE_APPEND: borra lo que había)
file_put_contents(FICHERO, "línea nueva" . PHP_EOL, LOCK_EX);
```

Flag	Significado
`FILE_APPEND`	Añade al final en vez de sobrescribir. **Sin este flag se borra todo el contenido anterior.**
`LOCK_EX`	Bloquea el fichero mientras se escribe, para que dos usuarios a la vez no mezclen datos.

Se combinan con el operador `|`: `FILE_APPEND | LOCK_EX`. Importante el `PHP_EOL` al final de cada línea: sin él, la siguiente línea se pega a la anterior.

3.2 La forma larga: fopen, fwrite, fclose

Útil cuando escribes varias veces seguidas o quieres controlar el puntero. Siempre sigue los mismos tres pasos: **abrir -> usar -> cerrar**.

```
$f = fopen(FICHERO, 'a');                       // 1. abrir en modo "añadir"
if ($f === false) {                             //    fopen devuelve false si no puede abrir
    throw new RuntimeException('No se pudo abrir el fichero.');
}
try {
    if (fwrite($f, $linea . PHP_EOL) === false) {   // 2. escribir (fputs es sinónimo)
        throw new RuntimeException('Error al escribir.');
    }
} finally {
    fclose($f);                                 // 3. cerrar SIEMPRE, aunque haya error
}
```

Modos de apertura de fopen

Modo	Lee	Escribe	Si NO existe	Si existe / puntero
`r`	Sí	No	Falla (false)	Puntero al principio
`r+`	Sí	Sí	Falla	Puntero al principio; sobrescribe desde ahí
`w`	No	Sí	Lo crea	**Lo vacía** (trunca a 0 bytes)
`w+`	Sí	Sí	Lo crea	**Lo vacía**
`a`	No	Sí	Lo crea	Puntero al final: **añade sin borrar**
`a+`	Sí	Sí	Lo crea	Escribe siempre al final; se puede leer
`x` / `x+`	No / Sí	Sí	Lo crea	Si existe, falla (no pisa nada)

> **TRAMPA:** El error más caro con ficheros: abrir con `'w'` para "añadir". `'w'` **borra** todo lo guardado antes. Para guardar registros nuevos: `'a'` (o `FILE_APPEND`).

3.3 Escribir varias líneas / reescribir el fichero entero

```
$lineas = ['1;España;Madrid', '2;Portugal;Lisboa', '3;Italia;Roma'];
file_put_contents(FICHERO, implode(PHP_EOL, $lineas) . PHP_EOL, LOCK_EX);    // pisa todo el fichero

// Si el array puede estar vacío, no escribas un salto de línea suelto:
$contenido = $lineas ? implode(PHP_EOL, $lineas) . PHP_EOL : '';
file_put_contents(FICHERO, $contenido, LOCK_EX);
```

4. Leer ficheros

4.1 La forma corta: file()

`file()` lee el fichero completo y devuelve un **array con una línea en cada posición**. Con los dos flags de abajo el resultado es limpio: sin `\n` al final y sin líneas vacías.

```
$lineas = file(FICHERO, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
if ($lineas === false) {
    throw new RuntimeException('No se pudo leer el fichero.');
}
// $lineas = ['1;España;Madrid', '2;Portugal;Lisboa', '3;Italia;Roma']
echo count($lineas);                  // 3 registros
echo $lineas[0];                      // 1;España;Madrid
foreach ($lineas as $numero => $linea) { echo "Línea " . ($numero + 1) . ": $linea<br/>"; }
```

> **OJO:** `file()` sobre un fichero que **no existe** da un warning y devuelve `false`. Comprueba antes con `is_file()`. Fichero vacío (existe pero sin líneas) devuelve un array vacío `[]`: no es un error.

4.2 Leer todo como texto: file_get_contents

```
$texto = file_get_contents(FICHERO);                    // string con todo el contenido (o false)
$lineas = explode("\n", str_replace("\r", '', $texto)); // equivalente a file() (con una última línea vacía)
echo nl2br(htmlspecialchars($texto));                   // mostrar un txt tal cual en HTML
```

4.3 Línea a línea: fopen + fgets

Recomendable para ficheros muy grandes (no carga todo en memoria) o cuando necesitas parar al encontrar algo.

```
$f = fopen(FICHERO, 'r');
if ($f === false) { throw new RuntimeException('No se pudo abrir el fichero.'); }
try {
    while (($linea = fgets($f)) !== false) {      // fgets devuelve false al llegar al final
        $linea = rtrim($linea, "\r\n");           // fgets INCLUYE el salto de línea: hay que quitarlo
        if ($linea === '') { continue; }          // ignorar líneas vacías
        echo htmlspecialchars($linea) . '<br/>';
    }
} finally {
    fclose($f);
}
```

> **TRAMPA:** Evita `while (!feof($f)) { $l = fgets($f); ... }`: la última vuelta recibe `false` y produce una línea fantasma. El patrón `while (($l = fgets($f)) !== false)` no tiene ese problema.

Otras funciones de lectura y del puntero

Función	Qué hace
`fgets($f)`	Lee una línea (con el salto de línea incluido).
`fgets($f, 100)`	Lee como máximo 99 caracteres.
`fgetc($f)`	Lee un solo carácter.
`fread($f, $n)`	Lee n bytes.
`feof($f)`	true si el puntero está al final del fichero.
`rewind($f)`	Vuelve el puntero al principio (para releer).
`ftell($f)` / `fseek($f, n)`	Posición actual del puntero / moverlo n posiciones (`SEEK_SET`, `SEEK_CUR`, `SEEK_END`).
`readfile($ruta)`	Lee el fichero y lo escribe directamente en la salida; devuelve el nº de bytes.

4.4 Una función de lectura reutilizable

Casi todos los ejercicios empiezan leyendo líneas. Tener **una** función bien hecha evita repetir comprobaciones:

```
/**
 * Lee un fichero de texto y devuelve sus líneas (sin saltos de línea ni líneas vacías).
 * Si el fichero no existe devuelve [] (aún no hay registros). Lanza excepción si no se puede leer.
 */
function leerLineas(string $ruta): array {
    if (!file_exists($ruta)) { return []; }
```

```
    if (!is_file($ruta) || !is_readable($ruta)) {
        throw new RuntimeException('El fichero no se puede leer.');
    }
    $lineas = file($ruta, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    if ($lineas === false) { throw new RuntimeException('Error al leer el fichero.'); }
    if ($lineas) {
        $lineas[0] = preg_replace('/^\xEF\xBB\xBF/', '', $lineas[0]);   // quita el BOM si lo hubiera
    }
    return array_map(fn($l) => rtrim($l, "\r"), $lineas);
}

/** Añade una línea al final del fichero. */
function agregarLinea(string $ruta, string $linea): void {
    if (file_put_contents($ruta, $linea . PHP_EOL, FILE_APPEND | LOCK_EX) === false) {
        throw new RuntimeException('No se pudo guardar el registro.');
    }
}

/** Sustituye el contenido del fichero por las líneas indicadas. */
function reescribirLineas(string $ruta, array $lineas): void {
    $texto = $lineas ? implode(PHP_EOL, $lineas) . PHP_EOL : '';
    if (file_put_contents($ruta, $texto, LOCK_EX) === false) {
        throw new RuntimeException('No se pudo actualizar el fichero.');
    }
}
```

5. De línea de texto a datos (y al revés)

5.1 explode e implode

```
$linea = 'José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46';
$c = explode(';', $linea);          // trocear por el separador -> array de 6 elementos
//   $c[0] nombre   $c[1] tipo   $c[2] fecha   $c[3] nº   $c[4] descuentos   $c[5] importe
echo $c[0];                         // José María Lucas
$n = (int)$c[3];                    // todo llega como TEXTO: convierte a número si vas a calcular
$importe = (float)$c[5];            // "26.46" -> 26.46   (el punto es el separador decimal de PHP)
$descuentos = explode('/', $c[4]);  // ['Abonado', 'Día del Espectador']  (segundo nivel de separador)

$linea2 = implode(';', [$c[0], $c[1], $c[2], $n, $c[4], number_format($importe, 2, '.', '')]);   // y al revés
```

Campo	Índice	Conversión al leer	Formato al guardar
Nombre	0	texto, sin cambios	`limpiarCampo($nombre)`
Tipo de entrada	1	texto	texto
Fecha dd/mm/aaaa	2	texto (o `DateTime::createFromFormat('d/m/Y', $c[2])`)	`$fecha->format('d/m/Y')`
Nº de entradas	3	`(int)$c[3]`	número entero
Descuentos	4	`explode('/', $c[4])`	`implode('/', $array)`
Importe	5	`(float)$c[5]`	`number_format($i, 2, '.', '')` (punto, sin miles)

DOS FORMATOS: Para **mostrar** un importe al usuario usa coma (`number_format($i, 2, ',', '.')` -> 1.234,50). Para **guardarlo** en el fichero usa punto y sin separador de miles, o `(float)` no lo entenderá al leerlo.

5.2 Validar que la línea está bien formada

```
$c = explode(';', $linea);
if (count($c) !== 6) {
    throw new UnexpectedValueException("Línea con formato incorrecto: $linea");   // o 'continue' para
        saltarla
}
```

5.3 Tres formas de representar un registro

```
// a) Array indexado: rápido pero poco legible  ->  $c[2]
// b) Array asociativo: más legible
[$id, $pais, $capital] = explode(';', $linea);            // desestructurar con nombres
$registro = ['id' => (int)$id, 'pais' => $pais, 'capital' => $capital];
$paises[(int)$id] = $registro;                            // y usar el id como clave para buscar directo

// c) Objeto de una clase (lo que piden en el examen): toLinea() para guardar y desdeLinea() para leer
class Entrada {
    public function __construct(private string $nombre, private string $tipo, private string $fecha,
                                private int $num, private array $descuentos, private float $importe) {}

    public function toLinea(): string {                  // objeto -> texto
        $d = $this->descuentos ? implode('/', $this->descuentos) : 'Ninguno';
        return implode(';', [$this->nombre, $this->tipo, $this->fecha, $this->num, $d,
                             number_format($this->importe, 2, '.', '')]);
    }
    public static function desdeLinea(string $l): Entrada {   // texto -> objeto (método estático)
        $c = explode(';', $l);
        if (count($c) !== 6) { throw new UnexpectedValueException("Línea incorrecta: $l"); }
        return new Entrada($c[0], $c[1], $c[2], (int)$c[3], $c[4] === 'Ninguno' ? [] : explode('/', $c[4]),
            (float)$c[5]);
    }
    public function getNombre(): string { return $this->nombre; }      // getters para pintar la tabla
    // ... resto de getters
}
```

La regla de oro: **lo que escribe `toLinea()` debe poder leerlo `desdeLinea()`**. Si cambias una, cambia la otra.

5.4 Alternativa: fputcsv / fgetcsv

PHP trae funciones para ficheros con campos separados (CSV). Son cómodas, pero `fputcsv` **encierra entre comillas los campos que tengan espacios**, así que el fichero no quedaría con el formato exacto que pide el examen. En ejercicios con formato impuesto usa `implode` y `explode`.

```
$f = fopen(FICHERO, 'a');
fputcsv($f, ['José María Lucas', 'General'], ';', '"', '');   // escribe: "José María Lucas";General
fclose($f);
$f = fopen(FICHERO, 'r');
while (($fila = fgetcsv($f, 0, ';', '"', '')) !== false) { /* $fila es un array de campos */ }
fclose($f);
```

6. Modificar, borrar y otras operaciones

Un fichero de texto no permite borrar o cambiar una línea "en el sitio". El método universal: **leer todo -> cambiar el array en memoria -> reescribir el fichero entero**.

6.1 Id autoincremental

```
function siguienteId(string $ruta): int {
    $max = 0;
    foreach (leerLineas($ruta) as $linea) {
        $max = max($max, (int)explode(';', $linea)[0]);       // el id es el primer campo
    }
    return $max + 1;                                          // fichero vacío -> 1
}
agregarLinea(FICHERO, siguienteId(FICHERO) . ';' . limpiarCampo($tarea) . ';0');
```

6.2 Borrar un registro por id

```
function borrarRegistro(string $ruta, int $id): bool {
    $nuevas = [];
    $borrado = false;
    foreach (leerLineas($ruta) as $linea) {
        if ((int)explode(';', $linea)[0] === $id) { $borrado = true; continue; }   // saltar la que se borra
        $nuevas[] = $linea;
    }
    if ($borrado) { reescribirLineas($ruta, $nuevas); }
    return $borrado;
}
```

6.3 Modificar un registro

```
function marcarHecha(string $ruta, int $id): void {
    $lineas = leerLineas($ruta);
    foreach ($lineas as $i => $linea) {
        $c = explode(';', $linea);
        if ((int)$c[0] === $id) { $c[2] = '1'; $lineas[$i] = implode(';', $c); }   // cambiar y volver a unir
    }
    reescribirLineas($ruta, $lineas);
}
```

6.4 Último registro, los N últimos, orden inverso

```
$lineas = leerLineas(FICHERO);
$ultima = end($lineas);                                  // la última línea (false si el array está vacío)
$ultimas5 = array_slice($lineas, -5);                    // las 5 últimas, en orden del fichero
$recientesPrimero = array_reverse($lineas);              // más reciente arriba
$total = count($lineas);                                 // cuántos registros hay
usort($lineas, fn($a, $b) => strcmp(explode(';', $a)[1], explode(';', $b)[1]));   // ordenar por el campo 1
```

6.5 Operaciones con ficheros y carpetas

Función	Qué hace
`file_exists($r)`, `is_file($r)`, `is_dir($r)`	Existe / es un fichero / es una carpeta.
`is_readable($r)`, `is_writable($r)`	Se puede leer / escribir (en una carpeta: se pueden crear ficheros dentro).
`filesize($r)`	Tamaño en bytes. `filesize($r) === 0` = fichero vacío.
`filemtime($r)`	Fecha de la última modificación (timestamp): `date('d/m/Y H:i', filemtime($r))`.
`copy($origen, $destino)`	Copia el fichero; devuelve true/false.
`rename($viejo, $nuevo)`	Renombra o mueve.
`unlink($r)`	Borra el fichero.
`mkdir($d, 0755, true)`	Crea la carpeta (true = crea también las carpetas intermedias).
`scandir($d)`	Array con los nombres de lo que hay en la carpeta (incluye `.` y `..`).
`glob('datos/*.txt')`	Array con las rutas que coinciden con el patrón.
`basename($r)`, `dirname($r)`, `pathinfo($r)`	Nombre del fichero / carpeta / partes de la ruta (nombre, extensión).

```
// Copia de seguridad antes de reescribir
if (is_file(FICHERO)) { copy(FICHERO, FICHERO . '.bak'); }
// Recorrer los .txt de una carpeta
foreach (glob(__DIR__ . '/datos/*.txt') as $ruta) { echo basename($ruta) . ' - ' . filesize($ruta) . '
    bytes<br/>'; }
```

7. Errores y excepciones con ficheros

Con ficheros las cosas fallan por motivos ajenos al código: el fichero no existe, no hay permisos, el disco está lleno, alguien editó el `.txt` a mano y dejó una línea rota. Un buen ejercicio **prevé** esas situaciones y avisa al usuario con claridad.

Situación	Qué hacer
El fichero aún no existe (primera ejecución)	Si se trata de datos guardados: **no es un error**, devuelve lista vacía (`if (!file_exists(...)) return [];`). La tabla se muestra vacía.
El fichero debe existir (por ejemplo `paises.txt`)	Lanza excepción y muestra un mensaje: 'No se encuentra el fichero de países'.
`fopen`, `file`, `file_put_contents` devuelven `false`	Son warnings, no excepciones: compruébalo con `=== false` y lanza **tú** la `RuntimeException`.
Línea con campos de menos	`count(explode(...)) !== N` -> ignora la línea (`continue`) o lanza `UnexpectedValueException`.
Fichero vacío	`file()` devuelve `[]`; comprueba `if (!$lineas)` antes de usar `array_rand` (daría error con un array vacío).
Permisos / carpeta inexistente	`is_writable` / `is_dir` + `mkdir`.

```
$errores = [];
try {
    $entradas = array_map(fn($l) => Entrada::desdeLinea($l), leerLineas(FICHERO));
} catch (RuntimeException $ex) {                // problemas de lectura
    $errores[] = 'No se pueden mostrar las entradas: ' . $ex->getMessage();
    $entradas = [];                             // que la página siga funcionando
} catch (UnexpectedValueException $ex) {        // línea mal formada
    $errores[] = 'El fichero contiene datos incorrectos.';
    $entradas = [];
}
// Mensajes al usuario: claros, sin rutas completas ni errores técnicos. Si hace falta,
    error_log($ex->getMessage()).
```

MALA PRÁCTICA: No uses `@` delante de las funciones para "callar" los warnings (`@file(...)`): esconde el problema. Comprueba y gestiona.

8. Ejercicio A: entradas vendidas (examen, ejercicio 3)

Enunciado: copia `Ejercicio2.php` a `Ejercicio3.php`. Crea la clase `Entrada` y úsala para guardar los datos de las entradas vendidas con este formato: `NombreCliente;Tipo;FechaEvento(dd/mm/yyyy);NºEntradas;Descuentos;Importe`, por ejemplo `José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46`.

- Al pulsar **Comprar**, si todos los datos son correctos, crear un objeto `Entrada` y guardarlo en el fichero.
- Mostrar una **tabla con todas las entradas guardadas**. Se muestra **siempre**, aunque esté vacía.

8.1 Cómo pensarlo

Pregunta	Respuesta
¿Qué guardo?	Una línea por compra, con los 6 campos en el orden del enunciado.
¿Cuándo guardo?	Solo si `validar()` no devuelve errores (la compra es correcta).
¿Cuándo leo?	**Después** de guardar, para que la compra recién hecha ya salga en la tabla.
¿Y si el fichero no existe?	Es la primera compra: se crea solo al guardar (`FILE_APPEND`) y, mientras tanto, la tabla sale vacía.
¿Qué pasa con los descuentos?	Se guardan unidos con `/`. Si no hay ninguno (o solo 'Sin descuento'), se guarda `Ninguno` y al leer se convierte en un array vacío.
¿Dónde está la clase?	En `Entrada.php`, con `toLinea()` (objeto -> línea) y `desdeLinea()` (línea -> objeto). El resto del programa no sabe cómo es la línea.

8.2 Código (parte de ficheros)

`Entrada.php` es la clase de la sección 5.3 completa (constructor, `toLinea`, `desdeLinea` y getters). Las funciones de ficheros, usando las del apartado 4.4:

```
<?php
// Ejercicio3.php  (arriba van las constantes, e(), recogerDatos(), validar(), calcularTotal() del ejercicio
    2)
require_once __DIR__ . '/Entrada.php';
const FICHERO = __DIR__ . '/entradas.txt';

/** Guarda una entrada al final del fichero. */
function guardarEntrada(Entrada $entrada): void {
    agregarLinea(FICHERO, $entrada->toLinea());
}

/** Devuelve un array de objetos Entrada con todo lo guardado ([] si no hay nada). */
function leerEntradas(): array {
    $entradas = [];
    foreach (leerLineas(FICHERO) as $linea) {
        try { $entradas[] = Entrada::desdeLinea($linea); }
        catch (UnexpectedValueException $ex) { /* línea corrupta: se salta, el resto sigue */ }
    }
    return $entradas;
}

// ----------------- Programa principal -----------------
$errores = [];  $ticket = null;  $entradas = [];
try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $datos = recogerDatos($_POST);
        $errores = validar($datos, $espectaculos);
        if (!$errores) {                                                    // 1) datos correctos
            $total = calcularTotal($datos['tipo'], (int)$datos['entradas'], $datos['descuentos']);
            $fecha = DateTime::createFromFormat('Y-m-d', $datos['fecha'])->format('d/m/Y');   // dd/mm/aaaa
            $entrada = new Entrada($datos['nombre'], NOMBRE_TIPO[$datos['tipo']], $fecha,
                                   (int)$datos['entradas'],
                                   array_values(array_diff($datos['descuentos'], ['Sin descuento'])), $total);
            guardarEntrada($entrada);                                       // 2) crear objeto y guardar
            // ... preparar $ticket como en el ejercicio 2
        }
    }
} catch (Throwable $ex) {
    $errores[] = 'No se pudo completar la compra: ' . $ex->getMessage();
}
try {
    $entradas = leerEntradas();                                             // 3) leer DESPUÉS de guardar
} catch (Throwable $ex) {
```

```
    $errores[] = 'No se pueden mostrar las entradas vendidas.';
}
?>
```

Y en el HTML, la tabla (siempre visible):

```
<h2>Entradas vendidas</h2>
<table border="1" cellpadding="4">
  <tr><th>Cliente</th><th>Tipo</th><th>Fecha</th><th>Nº</th><th>Descuentos</th><th>Importe</th></tr>
  <?php foreach ($entradas as $en): ?>
    <tr><td><?= e($en->getNombre()) ?></td><td><?= e($en->getTipo()) ?></td><td><?= e($en->getFecha()) ?></td>
        <td><?= $en->getNumEntradas() ?></td><td><?= e($en->getDescuentos()) ?></td>
        <td><?= number_format($en->getImporte(), 2, ',', '.') ?> &euro;</td></tr>
  <?php endforeach; ?>
  <?php if (!$entradas): ?><tr><td colspan="6">Todavía no se ha vendido ninguna entrada.</td></tr><?php
      endif; ?>
</table>
```

8.3 Traza de una compra

Momento	Contenido de entradas.txt	Tabla
Primera visita	(no existe)	Vacía + mensaje
Compra con datos incorrectos	(sin cambios)	No cambia: se muestran los errores
Compra correcta de José María (3 entradas, Abonado + Día del Espectador)	`José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46`	1 fila
Segunda compra correcta (Ana, sin descuentos)	Las dos líneas, la nueva al final	2 filas

9. Ejercicio B: juego de capitales

Enunciado: un fichero de texto con países y capitales con estructura `id;país;capital`. La página selecciona un país **al azar** (función `rand`) y pregunta por su capital. Mecánica:

Situación	Qué ve el usuario
Carga de la página	Título 'JUEGO CAPITAL', '¿Cuál es la capital de Portugal?', un campo 'Introduce capital' y los botones **Validar** y **Pasar**.
Validar con respuesta incorrecta	Sigue el **mismo país**, se mantiene lo escrito (madrid) y aparece 'Incorrecto' en **rojo**.
Validar con respuesta correcta	Sigue el mismo país y aparece 'Correcto' en **verde** (admite `lisboa` en minúsculas).
Pasar	Se elige **otro país** al azar y se vuelve a empezar (sin mensaje).

9.1 Cómo pensarlo

- **El problema clave**: HTTP no recuerda nada entre peticiones. Al pulsar Validar la página se recarga: ¿cómo sabe PHP de qué país era la pregunta? -> con un campo `<input type="hidden" name="id">` que viaja con el formulario. **El id viaja, la solución no** (así el usuario no puede ver la capital en el código fuente).
- Por eso el fichero tiene un `id`: se carga en un array indexado por id (`$paises[$id]`) y se busca directamente la pregunta que se está respondiendo.
- Dos botones submit con distinto `name` (`validar` y `pasar`): `isset($_POST['validar'])` o `isset($_POST['pasar'])` indica cuál se pulsó.
- Primera carga (no hay POST): sortear país nuevo. Validar: **conservar** el país del hidden. Pasar: sortear **otro**.
- Comparación tolerante: sin distinguir mayúsculas, sin espacios sobrantes y (mejor) sin tildes. Con `mb_strtolower` porque `strtolower` no entiende la `Á`.

9.2 Fichero paises.txt de ejemplo

```
1;España;Madrid
2;Portugal;Lisboa
3;Francia;París
4;Italia;Roma
5;Alemania;Berlín
6;Reino Unido;Londres
```

9.3 Solución completa: juego_capital.php

```
<?php
// juego_capital.php - Juego: adivinar la capital de un país elegido al azar
const FICHERO = __DIR__ . '/paises.txt';           // formato de cada línea: id;país;capital

/** Lee el fichero y devuelve [id => ['pais' => ..., 'capital' => ...]]. Lanza excepción si falla. */
function cargarPaises(string $ruta): array {
    if (!is_file($ruta) || !is_readable($ruta)) {
        throw new RuntimeException('No se encuentra el fichero de países.');
    }
    $lineas = file($ruta, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    if ($lineas === false) { throw new RuntimeException('No se pudo leer el fichero de países.'); }

    $paises = [];
    foreach ($lineas as $i => $linea) {
        if ($i === 0) { $linea = preg_replace('/^\xEF\xBB\xBF/', '', $linea); }   // BOM
        $c = explode(';', rtrim($linea, "\r"));
        if (count($c) < 3) { continue; }                                          // línea mal formada
        $paises[(int)$c[0]] = ['pais' => trim($c[1]), 'capital' => trim($c[2])];
    }
    if (!$paises) { throw new RuntimeException('El fichero de países está vacío.'); }
    return $paises;
}

/** Elige un id al azar (distinto de $excluir si hay más de un país). */
function paisAleatorio(array $paises, ?int $excluir = null): int {
    $ids = array_keys($paises);
    if ($excluir !== null && count($ids) > 1) { $ids = array_diff($ids, [$excluir]); }
    return $ids[array_rand($ids)];
}

/** Minúsculas, sin espacios extremos y sin tildes, para comparar respuestas. */
function normalizar(string $t): string {
```

```
    $t = mb_strtolower(trim($t), 'UTF-8');
    return strtr($t, ['á' => 'a', 'é' => 'e', 'í' => 'i', 'ó' => 'o', 'ú' => 'u', 'ü' => 'u']);
}

function e(string $t): string { return htmlspecialchars($t, ENT_QUOTES, 'UTF-8'); }

// ---------------- Programa principal ----------------
$error = '';  $mensaje = '';  $clase = '';  $respuesta = '';  $id = 0;
try {
    $paises = cargarPaises(FICHERO);
    $id = (int)($_POST['id'] ?? 0);                              // país de la pregunta anterior (si la hay)
    $valido = isset($paises[$id]);                               // ¿existe ese id? (el hidden se puede
        manipular)

    if (isset($_POST['validar']) && $valido) {                   // botón VALIDAR: mismo país
        $respuesta = trim($_POST['respuesta'] ?? '');
        if ($respuesta === '') {
            $mensaje = 'Escribe una capital.';  $clase = 'error';
        } elseif (normalizar($respuesta) === normalizar($paises[$id]['capital'])) {
            $mensaje = 'Correcto';  $clase = 'acierto';
        } else {
            $mensaje = 'Incorrecto';  $clase = 'error';
        }
    } elseif (isset($_POST['pasar']) && $valido) {               // botón PASAR: otro país
        $id = paisAleatorio($paises, $id);
    } else {                                                     // primera carga (o id no válido)
        $id = paisAleatorio($paises);
    }
} catch (Throwable $ex) {
    $error = $ex->getMessage();
}
?>
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Juego capital</title>
  <style>
    .error   { color: red;   font-weight: bold; }
    .acierto { color: green; font-weight: bold; }
  </style>
</head>
<body>
  <h1>JUEGO CAPITAL</h1>
  <?php if ($error): ?>
    <p class="error"><?= e($error) ?></p>
  <?php else: ?>
    <form action="" method="post">
      <h3>¿Cuál es la capital de <?= e($paises[$id]['pais']) ?>?</h3>
      <input type="hidden" name="id" value="<?= $id ?>">
      <input type="text" name="respuesta" placeholder="Introduce capital" value="<?= e($respuesta) ?>"
          autofocus>
      <input type="submit" name="validar" value="Validar">
      <input type="submit" name="pasar" value="Pasar">
    </form>
    <?php if ($mensaje): ?><p class="<?= $clase ?>"><?= e($mensaje) ?></p><?php endif; ?>
  <?php endif; ?>
</body>
</html>
```

9.4 Qué hace cada trozo

Parte	Por qué está ahí
`cargarPaises()`	Lee el fichero **una vez** y lo convierte en un array indexado por id. Si algo falla lanza excepción, que se captura en el programa principal y se muestra como mensaje.
`if ($i === 0) preg_replace(... BOM)`	Si el `.txt` se guardó con BOM, el primer id sería `"\xEF\xBB\xBF1"` y `(int)` daría 0: el primer país no se encontraría.
`count($c) < 3 -> continue`	Si alguien deja una línea a medias, se ignora en vez de romper el juego.
`$valido = isset($paises[$id])`	El campo hidden lo puede modificar el usuario: nunca te fíes sin comprobarlo.
`isset($_POST['validar'])` / `['pasar']`	Distingue qué botón se pulsó. Un botón submit con `name` envía su valor solo cuando se pulsa.
`paisAleatorio($paises, $id)`	En Pasar excluye el país actual para no repetirlo seguidamente.
`value="<?= e($respuesta) ?>"`	Mantiene lo que el usuario escribió tras Validar (como en la imagen: 'madrid').
`e()` en todo	Escapa lo que se imprime: el nombre del país viene de un fichero que podría editarse.

SI NO FUNCIONA: Si el juego muestra siempre el mismo país tras Validar, olvidaste el `hidden` o lo estás sobrescribiendo con un nuevo sorteo en el `else`. Si 'Pasar' no cambia nada, comprueba que ambos botones tienen `name`.

9.5 Ampliación: guardar los intentos en un fichero

Para practicar escritura **y** lectura en el mismo ejercicio: registrar cada validación en `intentos.txt` (`fecha;país;respuesta;resultado`) y mostrar estadísticas.

```
const INTENTOS = __DIR__ . '/intentos.txt';

function registrarIntento(string $pais, string $respuesta, bool $acierto): void {
    $linea = implode(';', [date('d/m/Y H:i:s'), limpiarCampo($pais), limpiarCampo($respuesta), $acierto ?
        'OK' : 'FALLO']);
    agregarLinea(INTENTOS, $linea);                                  // se llama justo después de comparar
}

function estadisticas(): array {
    $aciertos = 0;  $total = 0;
    foreach (leerLineas(INTENTOS) as $linea) {
        $c = explode(';', $linea);
        if (count($c) !== 4) { continue; }
        $total++;
        if ($c[3] === 'OK') { $aciertos++; }
    }
    return ['total' => $total, 'aciertos' => $aciertos,
            'porcentaje' => $total ? round($aciertos * 100 / $total, 1) : 0];
}
// echo "Aciertos: {$est['aciertos']} de {$est['total']} ({$est['porcentaje']} %)";
```

10. Prácticas propuestas

Ordenadas de menor a mayor dificultad. Las dos primeras llevan solución; en las demás tienes pistas. Intenta hacerlas antes de mirar.

Práctica 1 - Contador de visitas (con solución)

Una página que muestre cuántas veces se ha cargado. El número se guarda en `visitas.txt`.

```
<?php
const RUTA = __DIR__ . '/visitas.txt';
try {
    $visitas = is_file(RUTA) ? (int)file_get_contents(RUTA) : 0;       // si no existe, empieza en 0
    $visitas++;
    if (file_put_contents(RUTA, (string)$visitas, LOCK_EX) === false) {   // sin FILE_APPEND: sobrescribe
        throw new RuntimeException('No se pudo guardar el contador.');
    }
    echo "Esta página se ha visitado $visitas veces.";
} catch (Throwable $ex) {
    echo 'No se puede mostrar el contador.';
}
```

Práctica 2 - Libro de visitas (con solución)

Formulario con nombre y mensaje. Cada mensaje se guarda con la fecha. Debajo se muestran los **5 últimos**, el más reciente arriba.

```
<?php
const LIBRO = __DIR__ . '/libro.txt';          // fecha;nombre;mensaje  (+ funciones leerLineas,
    agregarLinea, limpiarCampo, e)
$errores = [];
try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $nombre  = limpiarCampo($_POST['nombre'] ?? '');
        $mensaje = limpiarCampo($_POST['mensaje'] ?? '');            // quita ; y saltos de línea del textarea
        if ($nombre === '' || $mensaje === '') { $errores[] = 'Nombre y mensaje son obligatorios.'; }
        else { agregarLinea(LIBRO, implode(';', [date('d/m/Y H:i'), $nombre, $mensaje])); }
    }
    $ultimos = array_reverse(array_slice(leerLineas(LIBRO), -5));    // últimos 5, más reciente primero
} catch (Throwable $ex) { $errores[] = 'Error con el libro de visitas.'; $ultimos = []; }

foreach ($ultimos as $linea) {
    [$fecha, $nombre, $mensaje] = array_pad(explode(';', $linea), 3, '');
    echo '<p><b>' . e($nombre) . '</b> (' . e($fecha) . '): ' . e($mensaje) . '</p>';
}
```

Práctica 3 - Lista de tareas con borrar (pistas)

- Fichero `tareas.txt` con `id;texto;hecha`. Usa `siguienteId()` (apartado 6.1) al añadir.
- Pinta cada tarea con enlaces `?borrar=ID` y `?hecha=ID`. Al recibirlos con `$_GET`, valida que sean números (`ctype_digit`) y llama a `borrarRegistro()` / `marcarHecha()` (apartados 6.2 y 6.3).
- Tras borrar, redirige con `header('Location: tareas.php'); exit;` para que F5 no repita la acción.
- Mejora: usa un formulario `method="post"` con botón para borrar, que es más correcto que un enlace GET.

Práctica 4 - Ranking del juego de capitales (pistas)

- Guarda `fecha;jugador;aciertos;fallos` cuando el jugador pulse un botón 'Terminar'.
- Lee todas las líneas, conviértelas en arrays asociativos y ordénalas con `usort` por aciertos descendente (`$b['aciertos'] <=> $a['aciertos']`).
- Muestra el top 5 con `array_slice`. Si el fichero no existe, muestra 'Aún no hay partidas'.

Práctica 5 - Inventario de productos (pistas)

- Fichero `productos.txt` con `codigo;nombre;precio;stock` (créalo a mano, con precios con punto decimal).
- Muestra una tabla con todos los productos, el **valor total del stock** (precio x stock sumado), el producto **más caro** y marca en rojo los de stock menor que 5.

- Añade un formulario de alta con validación: código único (recorre el fichero para comprobar que no existe), precio numérico mayor que 0 y stock entero mayor o igual que 0.

Práctica 6 - Visor de ficheros de texto (pistas)

- Formulario con subida de un `.txt` (`enctype="multipart/form-data"`). Comprueba la extensión con `pathinfo` y el tamaño.
- Muévelo a `subidas/` con `move_uploaded_file`, léelo con `file_get_contents` y muéstralo con `nl2br(htmlspecialchars($texto))`.
- Muestra también el número de líneas, de palabras (`str_word_count`) y el tamaño (`filesize`). Lista los ficheros subidos con `scandir`/`glob` y enlaces para verlos.

11. Errores típicos y lista de comprobación

11.1 Por qué no se guarda o no se lee

Síntoma	Causa habitual y solución
El fichero se vacía en cada guardado	Usas `'w'` en `fopen` o falta `FILE_APPEND`. Para añadir: `'a'` / `FILE_APPEND`.
Todas las líneas salen pegadas en una sola	Falta `PHP_EOL` al escribir. Ponlo al final de cada línea.
Salen líneas vacías o un registro 'fantasma'	Falta `FILE_SKIP_EMPTY_LINES`, o usas `while (!feof($f))`. Usa el patrón `while (($l = fgets($f)) !== false)`.
Warning: file(...): Failed to open stream	La ruta no existe (¿es relativa y cambió el directorio?). Usa `__DIR__`. Comprueba `is_file()` antes.
Permission denied al escribir	Carpeta o fichero sin permisos de escritura, o fichero abierto/bloqueado por otro programa.
El primer registro no se reconoce	El fichero tiene BOM (UTF-8 con firma). Guárdalo como UTF-8 sin BOM o quítalo al leer.
Tildes mal (Ã³, ■)	Fichero no guardado en UTF-8 y/o falta `<meta charset="UTF-8">`.
Las comparaciones fallan aunque 'parece igual'	Espacios o `\r` al final de la línea: usa `trim`/`rtrim`; compara sin mayúsculas ni tildes con `normalizar()`.
Un número sale como 0 o un decimal se corta	Guardaste con coma (`26,46`): `(float)` solo entiende el punto. Guarda con `number_format($n, 2, '.', '')`.
Después de guardar la tabla no muestra lo nuevo	Estás leyendo **antes** de guardar. Lee después.
Se duplica el registro al pulsar F5	Se reenvía el POST. Redirige a la misma página tras guardar (`header('Location: ...'); exit;`) si el enunciado no pide recordar datos.
Un registro rompe la tabla	El usuario escribió `;` o un salto de línea. Usa `limpiarCampo()` antes de guardar.
`array_rand()` da error	Array vacío (fichero vacío o con líneas mal formadas). Comprueba `if (!$paises)` antes de sortear.

11.2 Lista de comprobación

- La ruta del fichero está definida **una vez** como constante y con `__DIR__`.
- Escribo con `FILE_APPEND | LOCK_EX` (o `'a'`) y cada línea termina en `PHP_EOL`.
- Compruebo `=== false` en `fopen`, `file`, `file_get_contents` y `file_put_contents`, y lanzo excepción con un mensaje claro.
- Si el fichero puede no existir todavía, devuelvo lista vacía en vez de dar error.
- Al leer elimino saltos de línea y líneas vacías, y compruebo que cada línea tiene el número de campos esperado.
- Convierto los tipos al leer (`(int)`, `(float)`) y guardo los decimales con punto.
- Limpio los textos libres (`;`, saltos de línea) antes de guardarlos y escapo con `htmlspecialchars` al mostrarlos.
- Cierro lo que abro (`fclose`, preferiblemente en `finally`).
- Leo **después** de guardar si la página debe mostrar lo recién guardado.
- La tabla/listado se muestra siempre, con un mensaje cuando no hay datos.
- Pruebo: primera ejecución sin fichero, fichero vacío, una línea rota a mano, tildes y datos con `;`.